Using LLMs to Build Gurobi Models In Record-Breaking Time

Michael Khalfin

July 13, 2026

Abstract

This paper introduces a novel framework for accelerating mathematical optimization modeling by translating human-readable Pyomo formulations into high-performance, Gurobi-native C-API structures. To overcome the slow construction times of traditional algebraic modeling languages, the authors deploy a large language model within an agentic workflow to generate a deterministic Python transpiler. To guarantee the correctness of each individually translated model, the system employs data-parametric verification, using differential testing on truncated datasets to ensure every transpiled code constructs a solver-level model that is structurally identical to its Pyomo reference. Computational benchmarks across canonical optimization problems demonstrate that this transpilation pipeline reduces model build times by orders of magnitude compared to standard Pyomo workflows. Finally, the framework's real-world efficacy is demonstrated through its successful deployment on a large-scale epidemic model.

1 Introduction

Mathematical programming is typically organized around three tasks: modeling, solving, and interpreting results. In the modeling stage, informal requirements and constraints are translated into mathematical objects such as sets, variables, parameters, constraints, and objective functions. Algebraic Modeling Languages (AMLs) provide a software layer for expressing these objects independently of solver-specific details. Pyomo is a widely used AML embedded in Python, allowing users to formulate optimization models with readable Python syntax while retaining access to a broad range of solvers (e.g., CBC, GLPK, IPOPT, CPLEX, or Gurobi) [10, 5]. For example, a capacity-constrained supply model can be written in Pyomo as follows:

```python
import pyomo.environ as pyo

data = {
    'Plants': ['Plant_A', 'Plant_B'],
    'Markets': ['Market_1', 'Market_2', 'Market_3'],
    'Supply': {'Plant_A': 100, 'Plant_B': 150}
}

def build_pyomo_model(data):
    m = pyo.ConcreteModel()
    m.Plants = pyo.Set(initialize=data['Plants'])
    m.Markets = pyo.Set(initialize=data['Markets'])
    m.x = pyo.Var(m.Plants, m.Markets, domain=pyo.NonNegativeReals)
    m.Supply = pyo.Param(m.Plants, initialize=data['Supply'])

    def supply_rule(m, i):
        return sum(m.x[i, j] for j in m.Markets) <= m.Supply[i]
    m.supply_constr = pyo.Constraint(m.Plants, rule=supply_rule)

    def supply_sum(m):
        return sum(m.x[i, j] for i in m.Plants for j in m.Markets)
    m.TotalVolume = pyo.Objective(rule=supply_sum, sense=pyo.maximize)

    return m
```

Following the execution of `build_pyomo_model(data)`, Pyomo constructs a *ConcreteModel*, an in-memory object-oriented representation of the mathematical program. Pyomo sequentially evaluates the provided rules, mapping sets and parameters to decision variables to construct a symbolic algebraic expression tree.

Then, when a user calls a solver, Pyomo must translate its rich expression tree into a format the target solver understands. To handle this translation for Gurobi, Pyomo provides specialized backends. While older pipelines relied on writing intermediate text files (e.g., LP or NL formats) to disk, modern implementations bypass disk I/O entirely using Python bindings. Two of these interfaces are `gurobi_direct`, which generates a one-time in-memory Gurobi model, and `gurobi_persistent`, which maintains the solver object in memory to facilitate repeated solves without rebuilding.

However, for large-scale problems with millions of decision variables, Pyomo's model construction and solver translation steps can become a bottleneck, taking hours or exceeding available memory. Solver-native APIs such as Gurobi's `addMVar` and matrix operations avoid much of this overhead by constructing sparse numerical representations directly, but they are typically less readable and harder to modify than AML formulations.

Thus, the standard workflow involves a tradeoff between modeling convenience and construction speed. Recent work on Pyomo `templatization` reduces model-generation overhead, but it is incompatible with the `gurobi_persistent` backend. As a result, the standard workflow in Figure 1 forces users to choose between faster initial construction and faster iterative re-solving.

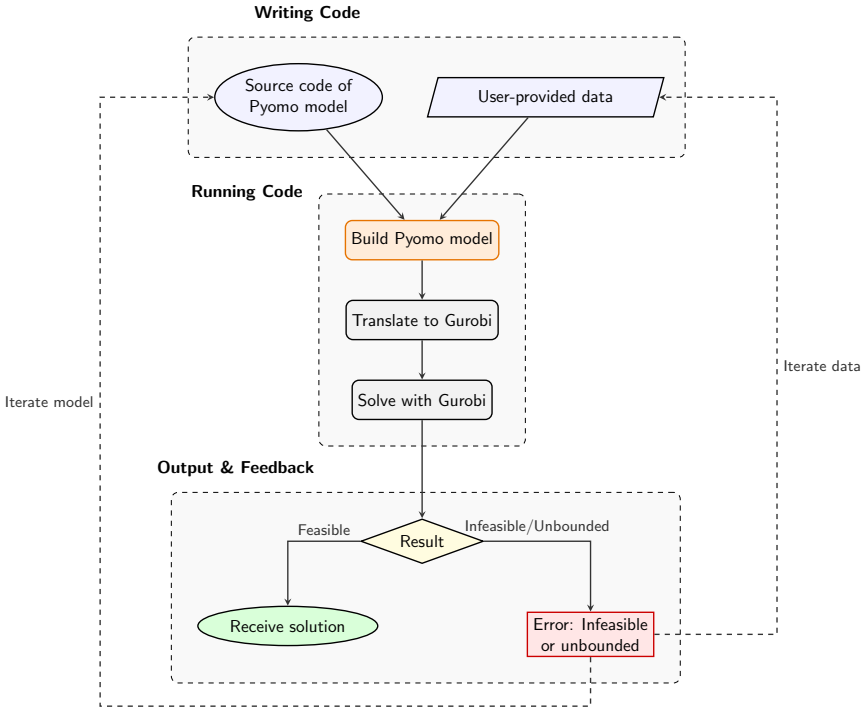

Figure 1: The workflow that Pyomo assumes by default.

This workflow has an additional drawback: the user may only discover infeasibility or unboundedness after the Pyomo model has already been built and translated. If the formulation or data must then be revised, the expensive construction process is repeated.

A more efficient workflow is shown in Figure 2. The Pyomo formulation remains the canonical, readable specification, but it is immediately *transpiled* into Gurobi-native model-building code rather than instantiated first.[1] This enables faster iteration when the model is infeasible or unbounded. Once feasibility is established, the Pyomo model can be instantiated retroactively to access the benefits of the modeling layer.

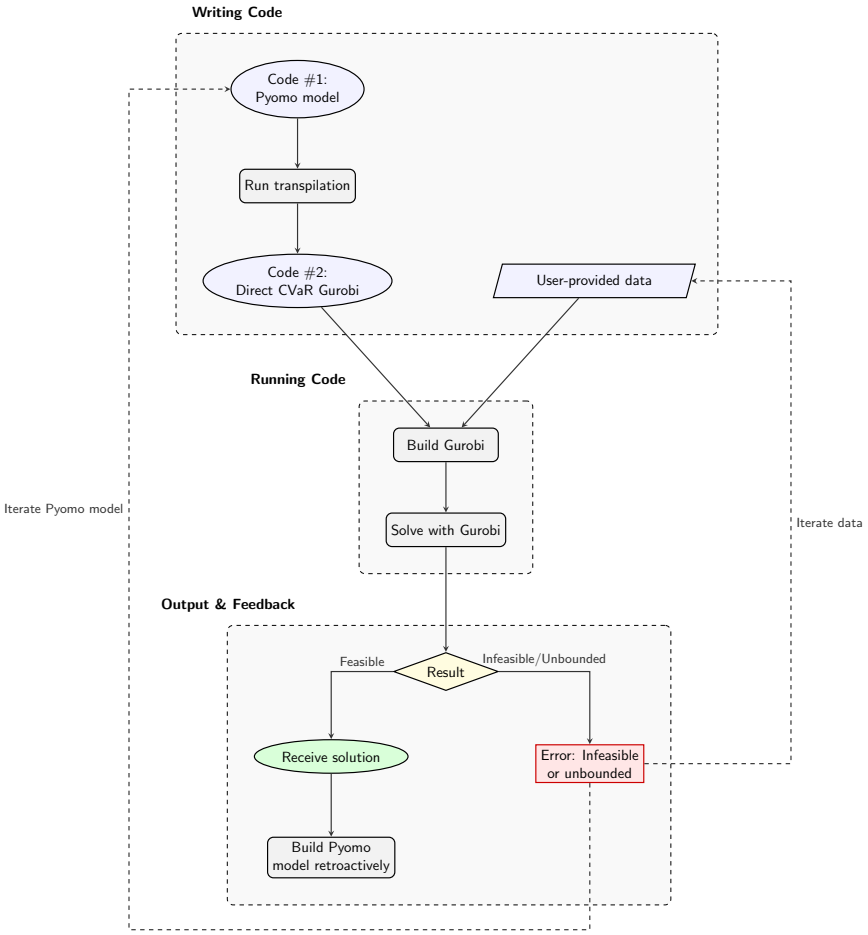

Figure 2: The Pyomo-to-Gurobi workflow developed in this paper, which preserves Pyomo as the canonical specification while enabling direct Gurobi-native model construction.

[1]By transpilation, we mean source-to-source translation: converting Pyomo model-building code into Python code that constructs the corresponding Gurobi model directly.

The central idea of this paper is that a frontier large-language model (LLM), used through an agentic coding workflow, can generate and refine a deterministic Python transpiler that converts human-readable Pyomo formulations into Gurobi-native model-building code. As shown in Figure 3, the stochastic component is confined to development of the transpiler. Once generated, the transpiler is an ordinary Python artifact: deterministic in execution, inspectable, testable, benchmarkable, and repairable.

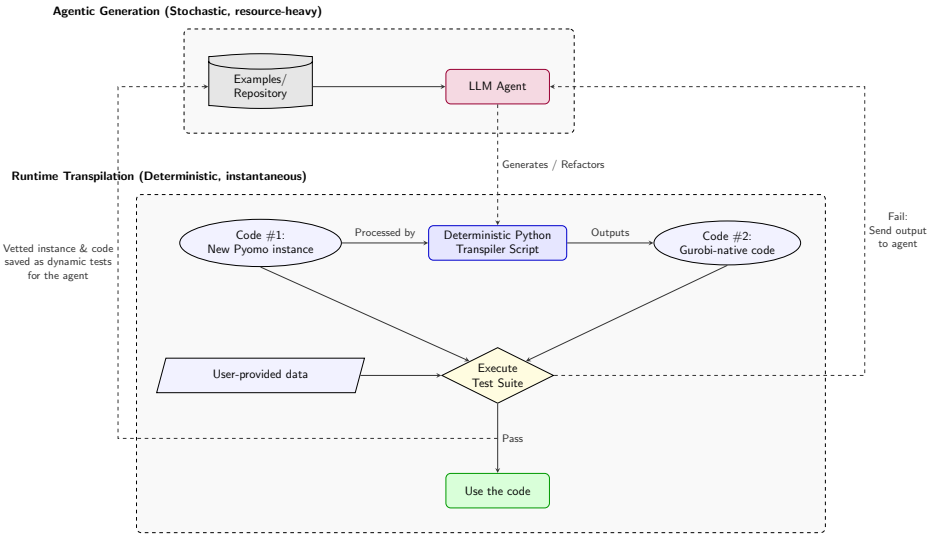

Figure 3: Development and verification of the transpiler using an agentic loop. Note that the 4 process nodes are the same as in Figure 2.

Its outputs are vetted by differential testing. For a given formulation, both the original Pyomo code and the transpiled Gurobi-native code are executed on the same data, and the resulting solver-level model structures are compared. Because the data is an explicit input, this validation is computationally inexpensive: we can repeatedly test on truncated datasets that preserve the full problem's indexing structure while keeping Pyomo's instantiation quick. If the two implementations construct identical models across multiple reduced instances, it becomes highly unlikely that the transpiler is merely coincidentally correct. The same deterministic code can then be run at full scale.

This approach provides a different trust model from much of the recent literature on LLMs for optimization. Rather than treating the LLM as a black-box optimizer or an oracle for formulation quality, we deploy it to produce an intermediate software artifact whose behavior can be validated and iteratively repaired. Failed tests are returned to the agent, while passing cases are retained as regression tests. Although we focus on Pyomo-to-Gurobi transpilation, the methodology generalizes across the modeling ecosystem: LLM-assisted transpilers can translate between any AMLs, solver APIs, or optimization transformations. More broadly, this pattern of data-parametric validation applies whenever a trusted reference implementation exists and is highly dependent on input data. By validating equivalence on reduced instances, developers can confidently deploy deterministic, agent-generated code at scale.

1.1 Contributions

In summary, this paper makes the following contributions:

1. **Agent-Generated Transpilation**: We demonstrate that an agentic LLM workflow can successfully construct a deterministic Python transpiler. This artifact bridges the abstraction gap, automatically translating human-readable Pyomo models into Gurobi-native C-API structures to achieve massive build-time speedups.

2. **Data-Parametric Verification**: We formalize a differential testing suite that leverages reduced datasets to strictly verify the transpiler's output against the original Pyomo model. This data-truncated validation mirrors the rigorous vetting workflows standard in national laboratory and industrial settings, showing algorithmic correctness before scaling to full production instances.

3. **The Oracle-Sandbox (OS) Methodology**: We formalize a generalized blueprint for bridging structural software gaps. In this paradigm, a stochastic agent is permitted to freely synthesize and iterate code (the *sandbox*), provided its outputs are strictly bounded by an independent verification mechanism (the *oracle*). The verification mechanism can be data-parametric testing, as used for our transpiler, or formal software provers like Rocq. This framework effectively functions as an operating system for natural-language programming: human intent (English) is "compiled" into deterministic software, with the oracle ensuring the correctness of the underlying implementation.

   We establish that this pairing of agent-driven generation with validation is particularly powerful for translation tasks. When a translation is required between any two rigorous representations, an agent can safely attempt to synthesize the missing bridge. We think that this methodology is applicable across a vast array of computational and engineering domains.

2 Background: LLMs in OR and Code Generation

LLMs in OR. Recent papers increasingly use LLMs to formulate mixed-integer linear programming problems directly from natural language descriptions. For example, the NL4Opt competition challenged participants to generate intermediate representations of linear programs from text for use by commercial solvers [18], building on earlier efforts to simplify the OR modeling experience [17]. This has spurred the creation of dedicated optimization benchmarks [23] and a variety of generative approaches. Multi-agent frameworks such as OptiMUS [1, 2] and others [22] parse complex natural language descriptions into programmatic constraints and objectives, while recent training-free architectures like SolverLLM leverage test-time search algorithms to iteratively guess and refine formulations [15]. Due to elevated computational costs and privacy concerns of frontier models, efforts like [11] have even trained open-source LLMs specifically for this task. A related stream of research deploys LLMs as interactive AI assistants to query model states [14] and debug infeasible Pyomo formulations [7]. These tools are primarily aimed at making optimization modeling accessible to non-experts, either by generating formulations from natural language or by supporting conversational interaction with existing models. Our focus is different. We assume that the algebraic model itself remains the rigorous source of truth, and ask how developers can preserve that exact, inspectable representation while making model construction and solver interaction substantially faster.

Beyond formulation, recent work has even applied LLMs to the algorithmic solving phase of MILPs [12, 20, 24], including heuristic tasks such as cold-start cutting plane configuration [13]. These methods operate downstream of model construction and are therefore complementary to our framework.

LLMs in Code Generation and Transpilation. A more direct antecedent to our work is LLM-assisted code transpilation: translating code from one language or representation into another. Recent systems have used LLMs for verified lifting into domain-specific languages [4], C-to-Rust migration with iterative repair [9], transpilation combined with program repair [19], and cross-language translation among general-purpose languages such as C, C++, Java, and Python [16]. These works highlight both the promise and the risk of LLM-generated code: the generated artifact can often be useful, but must be checked, repaired, or otherwise validated. While open-ended application synthesis typically relies on heuristic, LLM-based evaluators to judge the generated code [8], transpilation demands a vastly different trust model.

We enforce strict, deterministic validation in an optimization modeling context, using Pyomo as the reference implementation and Gurobi-native code as the transpilation target.

Our implementation uses Claude Code (powered by Claude Opus 4.8) as an agentic coding environment. Recent reports suggest that such systems can be effective on compiler-scale software projects. Notably, parallel Claude agents successfully authored a Rust-based C compiler through iterative task selection, testing,

and repair [6]. These long-running agentic workflows excel when grounded in deterministic tests, compact feedback, and rich repository context. In our framework, this context explicitly includes the source code of the Pyomo library itself, which embodies substantial prior work in algebraic modeling. Providing the agent with direct access to Pyomo's internal architecture makes the task more constrained than open-ended generation settings, where the model must invent premises or process steps from ambiguous information [25]. Here, the source program already exists, the target API is well defined, and the desired behavior can be checked against Pyomo. The LLM constructs a compiler-like translation path from one exact representation to another.

Beyond benchmarks and context, successful agentic generation relies on human-in-the-loop planning. Anthropic's analysis of Claude Code usage indicates that domain expertise remains critical for specifying architectural objectives and recognizing correct outcomes [3]. As detailed in Section 3, our workflow relies on active human prompting to direct the agentic loop. We provide the requisite optimization expertise—both by authoring the canonical Pyomo source models and by continually guiding the agent's objectives—while Claude Code executes the software engineering required to construct and refine the transpiler.

3 Methods: Agentic Generation of the Transpiler

Section 3.1 describes the agent's environment and context; Section 3.2 deep-dives into the OS method and how we applied it to build the transpiler iteratively. Section 3.3 presents the differential-testing framework that certifies each transpiled model, and Section 3.4 describes the transpiler it produced.

3.1 The Agentic Environment and Context

The transpiler was engineered within an agentic coding environment primarily driven by Claude Opus 4.8, supplemented by Claude Sonnet 4.6 for targeted refactoring and Claude Fable 5 for code reviews. The agent operates inside the project repository through an autonomous read–edit–run–observe cycle: it reads a file, changes it, executes the result, and inspects the output before the next step. To isolate development during trial-and-error phases, the agent operates within a dedicated Git branch, generating code changes locally and proposing formal software patches only when a stable iteration of the transpiler passes all verification checks. To manage the extensive repository history, the agent utilized a 1.0-million token context window with automatic context compaction enabled. Furthermore, development cycles were initiated using the environment's "Plan Mode," forcing the agent to explicitly map out its architectural strategy and align with task objectives before generating any code. Table 1 inventories what the agent could read and do.

Table 1: The resources available to the agent in the repository.

Reference Libraries	Pyomo source code files (local environment installation); `gurobipy-pandas` and `gurobipy` API documentation (read-only)
Repository Artifacts	Transpiler source code; 21 canonical Pyomo models serving as the differential and regression testing library (read/write)
Testing & Performance	Differential testing suite; computational benchmarking scripts (read/write)
Actions	Workspace regex search; read files; edit source; run shell commands and Python scripts (build, solve, test, benchmark); read output and error logs; persistent cross-session state memory

One property of this setup separates it from open-ended code generation: both endpoints of the translation path are fully specified *in the workspace*. The source language is Pyomo's own source code, and the target is bounded by the Gurobi API. The agent translates between two exact representations rather than inferring either of them. Because the Pyomo reference and the generated candidate share the repository, every emitted model can be executed and compared against ground truth on the spot (Section 3.3). The human contributes the domain-specific optimization models and architectural objectives, while the agent contributes the underlying software engineering [3, 6].

3.2 Task Decomposition and Prompting

Every programming paradigm rests on a foundational unit—the function in functional programming, the object in object-oriented programming—from which its idioms and guarantees follow. Programming by natural language rests on *generation*, and generation alone yields software that is fluent but unverified: convenient to produce, yet unsafe to trust at scale. This subsection concerns the discipline that closes that gap—how the agent is prompted, and why an expert programmer's reflexes can work against it.

The natural way to enlist a language model in a programming task is *imperative*: the operator, typically an experienced developer, prescribes the procedure. For example, they may delineate which abstract-syntax-tree nodes to visit, which pattern to match, how to structure the recursion. We found this mode counterproductive, as prescribing the procedure collapses the space of acceptable programs to a single human-conceived trajectory that a stochastic model must then reproduce exactly. A single deviation derails it, and the operator's own algorithmic imagination becomes the ceiling on what can be built. Paradoxically, fluency in software engineering can be a liability in this mode, precisely because it invites such prescription. The Oracle–Sandbox methodology inverts this interface. The operator specifies only *what* a correct output must satisfy—a set of declarative invariants and a mechanical test of membership—and never *how* to achieve it. The agent searches the space of programs, and it is the search, not the operator, that does the work.

Invariants, not procedures. For the transpiler, the human-supplied specification was a handful of algebraic correspondences—a Pyomo `Set` maps to a pandas index, a `Param` to a column, a summation over a set to a grouped relational reduction, a subset restriction to a join—together with the requirement, made precise in Section 3.3, that the emitted Gurobi model be structurally isomorphic to the one Pyomo builds. The parsing logic was never dictated, and this matters because the parsing logic is where the genuine difficulty lay.

Two summations that read almost identically in Pyomo compile to entirely different constructions. One aggregates a variable over one of its *own* index dimensions: the total flow on the arcs leaving a node, $\sum_j x_{nj}$. The other aggregates a variable against a *separate* mapping between indices: the shift start-times that cover a given hour, $\sum_{t' \in \mathrm{Valid}(t)} x_{t'}$. The only reliable signal separating them is whether the summation's loop index is itself one of the summed variable's indices or an external key that maps into them. Yet, the distinction has no surface syntactic marker. Sparse relations force further subtleties: a constraint may iterate over a three-component tuple (f, w, s) while the decision variable references only (f, s), requiring the generated code to retain the full tuple to enumerate the relation yet project onto a subset to align the join.

An operator prompting imperatively would have to foresee every such case and encode it by hand. Here no one did. The isomorphism invariant rejected each incorrect guess, and the agent iterated against that signal until the structures matched.

Decomposition into discriminating instances. This recasts what *decomposition* means. The task was not broken into coding steps but into *discriminating instances*. Each canonical model was chosen to exercise a structural case the transpiler did not yet handle, and its addition to the test suite defined the next target for the loop to reach. Recognizing that an equality whose right-hand side sums a projected tuple constitutes a genuinely new shape is an act of modeling judgment, not software engineering. It is exactly the contribution the domain expert is positioned to make.

The oracle co-evolved with this catalog. It began as a bare check that the two models had equal variable and constraint counts. As that weak test let defects through, it was progressively strengthened toward the permutation-invariant signature of Section 3.3. Each strengthening surfacing a class of error the looser test had silently passed, such as a dropped bound or a coefficient attached to the wrong variable.

Synthesis as a sequence of feasibility problems. Viewed through this lens, the methodology mirrors iterative methods familiar throughout operations research. The oracle defines a feasible region in the space of programs: those whose outputs satisfy every invariant. Against a *fixed* region the inner loop is ordinary feasibility-seeking. For any candidate program, the oracle returns a *localized residual*: the specific invariant

that failed. The agent takes a step that reduces this residual, the oracle re-evaluates, and a candidate eventually reaches the feasible region without any single step being prescribed or the operator knowing the path.

But the region is not fixed; it is refined in both directions. When the oracle admits a defective program, the operator adds a discriminating instance or strengthens the mathematical test. The test is a cut that contracts the region to exclude the newly exposed failure. Conversely, when the oracle rejects a program that is in fact correct, the operator relaxes the test. The equality-orientation and negative-zero normalizations detailed in Section 3.3 are exactly such relaxations, readmitting models that differ from the reference only cosmetically.

This is *constraint generation* paired with its converse—cuts separated to exclude what should be infeasible, and overly tight constraints dropped to admit what should be feasible—run until the region's boundary coincides with correctness itself. The software specification is not stated in full at the outset and then met. It is *discovered*, one cut at a time, as candidates reveal where the boundary was still misplaced.

Escaping a basin. A single oracle confines the search to a single basin. Correctness certifies behavior, not efficiency, and no accumulation of correctness-preserving repairs will lift a slow design out of its architecture. Escaping this local optimum demanded a *second* oracle: the benchmark of Section 4.1.2, which scores speed rather than correctness. Reaching the region this new oracle rewards meant changing the representation wholesale, shifting from building the model as pandas group-by reductions to assembling it as sparse coordinate matrices.

The repository still holds both transpilers this produced, `translator_old.py` and `translator.py`, which are of comparable size (roughly 1,900 and 2,300 lines). The second is not a patch of the first, but an entirely different program, certified by the same correctness oracle yet selected by a different objective. A loop closed on correctness alone would never have found it. That the search can afford such moves at all is a matter of cost. Each evaluation is a differential test on reduced instances completing in milliseconds (Section 3.3), attempted on an isolated branch and discarded for free when wrong, allowing the loops to run thousands of times.[2] Robustness in a difficult, high-dimensional space is bought not by one well-chosen step, but by many cheap, verified ones.

Where knowledge is created. It is worth asking where, in this loop, new knowledge actually enters (Figure 4). External sources (the Pyomo and Gurobi codebases) supply the definitions of the two endpoints; the operator supplies intent; the agent reads both and emits code; the code, once run, informs the operator. With these nodes alone the loop is *conservative*: information is transported and transformed, but nothing new is manufactured. It is the same closed circulation in which repeated exchanges, however expert the prompting, create nothing that was not already present.

The generative element is a further node: the *task*, embodied concretely as the oracle together with its growing catalog of verified instances. Unlike the others it is not a conduit but an accumulator. Every pass deposits into it—a new discriminating instance from the operator, a newly certified model from the agent—and both parties consult it on the next pass. Because it grows, the loop is a ratchet rather than a cycle: each turn leaves the shared specification more complete than before. The missing ingredient in an unstructured human–model loop is not a larger model or a cleverer prompt, but a task permitted to become a participant (to change and grow) rather than a fixed backdrop.

3.3 Verification by Differential Testing

The transpiler is trusted only because each model it emits is checked against Pyomo every time. Establishing that two Gurobi models coincide is subtler than a field-by-field comparison: the pipelines emit variables and constraints in different orders and under different names, so equality holds only up to a permutation of rows

[2]We refer here to the marginal financial and temporal cost of trial-and-error from the developer's perspective. While the global computational footprint of frontier LLMs is undeniably massive, the financial cost of a failed generation in a standard flat-rate consumer environment (e.g., Anthropic's subscription model) is effectively zero.

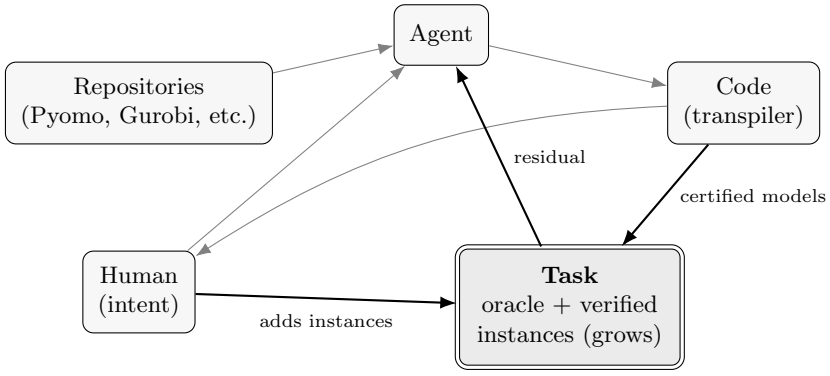

Figure 4: Knowledge flow in the loop. Along the thin arrows information is only transported: a conservative circulation in which nothing new is created. New knowledge accumulates only in the *task* (double border): every pass deposits into it, and both parties consult it on the next. Because the task grows, the loop is a ratchet, not a cycle.

and columns. Cast each model as the bipartite incidence graph between its variables and its constraints—an edge, weighted by the coefficient a_{uv}, wherever variable v appears in constraint u—and the two models agree exactly when these labelled graphs are isomorphic.

We test this isomorphism efficiently by computing a permutation-invariant *signature* in two layers (Table 2). The first layer deals with both nodes and edges, but ignores how they connect. It consists of order-independent multisets, such as global counts, coefficient values, variable bounds, and right-hand sides. A mismatch at this layer instantly flags a defect and isolates the specific parameter that the transpiler mishandled. However, these summaries cannot detect incidence errors, such as a correct coefficient being assigned to the wrong variable.

To capture this incidence structure, the second layer applies *color refinement* [21] (the 1-dimensional Weisfeiler-Leman algorithm). Each node is initialized with a "color" (a hash) encoding its intrinsic properties: a variable's bounds, type, and objective coefficient; or a constraint's sense and right-hand side. In each iteration t, every node is recolored by hashing its current color with the multiset of its neighbors' colors and the connecting edge weights (the constraint coefficients a_{uv}):

$$c_{t+1}(v) \;=\; \mathrm{hash}\Big(c_t(v),\; \big\{\,(c_t(u),\, a_{uv}) : u \in N(v)\,\big\}\Big), \tag{1}$$

Here, hash$(\cdot)$ denotes a deterministic, collision-resistant mapping that assigns a uniquely identifiable integer to each distinct input state. The process repeats until the color distribution stabilizes. Structurally interchangeable nodes converge to a common color, and the final histogram of colors perfectly characterizes the incidence graph. Two models are declared structurally equivalent only if both layers of their signatures match exactly.

To prevent false negatives, the signature normalizes two common algebraic symmetries. First, an equality row and its negation (e.g., $a^\top x = b$ versus $-a^\top x = -b$) state the identical constraint. Each equality is therefore encoded as the *unordered* pair of its positive and negative evaluations, ensuring that arbitrary negations hash to the same value. Second, floating-point artifacts such as a -0.0 right-hand side—which the transpiler's low-level arithmetic can occasionally yield where Pyomo yields 0.0—are strictly normalized before hashing.

Because the dataset is explicitly passed to the model function, this verification is executed on *reduced* instances. These instances preserve the exact indexing logic of the full problem but drastically shrink the domain sets, allowing Pyomo to instantiate the reference graph in milliseconds. We test each benchmark model at several sizes and three seeds—thirty instances—together with twenty-one formulations that each isolate one modeling feature. As the transpiler is deterministic, code certified on these instances is exactly the code that runs at full scale.

Table 2: The two layers of the structural signature and the discrepancy each detects. Every component is invariant to relabeling of variables and constraints.

Signature component	**Discrepancy it detects**
Counts (V, C, nnz)	Missing or extra variable, constraint, or term
Coefficient multiset	Wrong constraint coefficient
Right-hand-side multiset	Wrong right-hand side
Objective-coefficient multiset	Wrong objective weight
Bound/type multiset (ℓ, u, τ)	Wrong variable bound or domain
Color-refinement histogram, Eq. (1)	Wrong incidence: which variables enter which constraints

To ensure the signature is robust, we confirmed its sensitivity against deliberate, microscopic corruptions of a correct model (Table 3). During the agentic loop, a failing signature immediately isolates the mismatched component, providing localized feedback for the LLM's repair cycle. Once a generated model passes, it is permanently retained as a regression test, ensuring the transpiler's capabilities grow monotonically.

Table 3: Negative controls. Each corruption of a correct model changes the signature. Comparing variable and constraint counts alone misses three of the four.

Corruption	**First flagged by**	**Counts alone?**
Remove a constraint	Counts (C, nnz)	Yes
Tighten a variable bound	Bound multiset	No
Perturb one coefficient	Coefficient multiset, colors	No
Alter an objective coefficient	Objective multiset	No

The strict verification protocol also proved useful during development. For example, it successfully caught the semantic defect causing the transpiler to silently drop the upper bound in formulations where variables carried explicit interval bounds.[3]

3.4 The Transpiler

The transpiler operates as a deterministic static compiler. Rather than executing the Pyomo model and paying the overhead of instantiation, it parses the Python source code directly into an Abstract Syntax Tree (AST). The transpiler then translates this AST into an equivalent, high-performance Gurobi builder script. As illustrated in Figure 5, the pipeline mirrors traditional compiler architecture—parse, classify, reconcile indices, and generate. The parsing and generation phases are purely mechanical. Only the classification stage embeds domain-specific mathematical modeling knowledge.

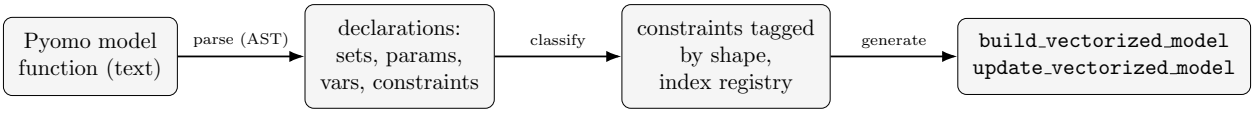

Figure 5: The transpiler as a compiler. Only the *classify* stage embeds modeling knowledge: the catalog of constraint shapes in Table 5. The output is Gurobi-native source.

Where the work runs. The speedup is a matter of *where* construction happens. Let V, C, and N denote the numbers of variables, constraints, and nonzero coefficients, respectively. Standard Pyomo instantiates the formulation by allocating at least one distinct Python object in memory for every variable, constraint,

[3]This left the model dimensions unchanged but enlarged its feasible region.

and expression term. This results in $\Theta(V+C+N)$ interpreted allocations before the model ever reaches the solver. Traversing millions of disjoint Python objects introduces severe cache misses, alongside a large per-object constant and its attendant memory traffic. For large-scale problems, these factors cause construction times to become superlinear in practice (Section 4.1.2).

Pyomo's templatization feature mitigates some of this by removing per-row expression trees, but it still incurs the $\Theta(V)$ overhead of allocating individual variable objects. The transpiler eliminates this Python-level instantiation entirely. Instead of creating objects per variable or constraint, the generated script issues an $O(1)$ number of vectorized library calls, completely independent of the problem size (Table 4). The $\Theta(N)$ work of assembling the constraint matrix is pushed entirely into compiled C-level array operations using pandas and SciPy. By maintaining the data in contiguous memory and passing sparse arrays directly to Gurobi's `addMVar` API, the computational cost strictly tracks nonzeros in compiled code rather than discrete component objects in interpreted code.

Table 4: Construction cost for a model with V variables, C constraints, and N nonzero coefficients. The pipelines share the $\Theta(N)$ order, but they differ in whether that work runs as interpreted object allocation or compiled array code, and in how many size-dependent Python-level operations they issue.

Pipeline	**Intermediate form**	**Python object allocations**	**Where $\Theta(N)$ runs**	**In-place re-solve**
Pyomo (`persistent`)	Expression tree per constraint	$\Theta(V+C+N)$	Interpreted	Yes
Pyomo + templatization	One template per block	$\Theta(V)$	Interpreted	No
Transpiler (COO+MVar)	Sparse COO arrays	$O(1)$	Compiled	Yes

From a rule to a matrix. The structural assembly relies on treating constraint generation as a relational join. Consider a standard supply constraint, $\sum_{j \in J} x_{ij} \leq s_i$. To map this to a sparse matrix, we number the variables using a bijection $\gamma : (i,j) \mapsto \{0, \ldots, |I||J|-1\}$ and the constraints using $\rho : i \mapsto \{0, \ldots, |I|-1\}$. The corresponding block of the coefficient matrix is $A \in \{0,1\}^{|I| \times |I||J|}$ with nonzeros located at $A_{\rho(i),\,\gamma(i,j)} = 1$.

The transpiler generates dataframes representing a table of variable columns $\{(i, j, \gamma(i,j))\}$ and a table of constraint rows $\{(i, \rho(i))\}$. An equi-join on the shared index i yields exactly the nonzero coordinates $(\rho(i), \gamma(i,j))$. These coordinates directly populate a sparse COO array passed to Gurobi using the method `addMConstr`$(A,\, x,\, \leq,\, s)$. For a weighted sum such as $\sum_j w_{ij} x_{ij}$, the parameter w_{ij} serves as the coefficient column in the join in place of 1.

Each constraint shape the transpiler recognizes corresponds to one such vectorized recipe (Table 5): a sequence of joins that produces the nonzero coordinates, a coefficient column, and a sparse assembly. The capacity, flow-conservation, and bill-of-materials constraints evaluated in Section 4.1 are all instances of these patterns.

Build and update. The transpiler emits its output as Python source text, which a single call to the built-in `exec` compiles into a live module. Until it is executed the output remains ordinary, human-inspectable source rather than an opaque in-memory object. The module exposes two functions. `build_vectorized_model` constructs the initial model, and `update_vectorized_model` overwrites the right-hand sides and objective coefficients of an existing model in $\Theta(N)$ time without touching its structure. This capability is paramount for workflows such as inference on past data or rapid scenario analysis, where the mathematical architecture is constant but the data parameters rotate frequently. Templatization forfeits this capability through its incompatibility with the persistent backend.

To guarantee this performance and maintain checkability, the transpiler is deliberately restricted to (mixed-integer) linear models whose constraints map to the known catalog (Table 5). Furthermore, we

Table 5: Representative constraint shapes and the sparse-assembly recipe each compiles to. The relation is $\leq$, $=$, or $\geq$; the objective, when present, is assembled the same way as a single row.

Shape	**Algebraic form**	**Assembly**
Grouped sum	$\sum_j w_{ij}\, x_{ij} \leq b_i$	Join variable table with row table on the shared index; coefficient w_{ij} (or 1)
Flow balance	$\sum_{\delta^+(n)} x - \sum_{\delta^-(n)} x = b_n$	Two joins on the out/in arc sets, coefficients $+1/-1$, concatenated
Cross-dimensional	$z_{ct} = \sum_{p \in P(c)} a_{pc}\, y_{pt}$	Join a with y on p, then with the row table on (c, t); coefficient a_{pc}; the z term contributes $-I$
Direct bound	$x_i \leq b_i$	One nonzero per row (identity-like)
Linear combination	$\sum_i (\alpha x_i + \beta y_i) \leq b$	One block per variable term, each with its own coefficient

enforce strict authoring conventions: index arithmetic must be precomputed into sets, and data-dependent branching within algebraic rules is prohibited. These constraints ensure that the static pattern-matching remains mathematically sound. The catalog of supported topological shapes was developed iteratively, with each newly generated pattern permanently locked in by the differential regression suite detailed in Section 3.3.

Crucially, this restriction bounds what the transpiler *can build*, not what it might silently get wrong. A construct outside the catalog is never mistranslated undetected: it either raises an exception during transpilation, or—if forced into the nearest pattern—produces a model that the differential test rejects as structurally distinct from the Pyomo reference (Section 3.3). No transpiled model is trusted until it passes that check. Because the readable Pyomo formulation is always retained as the canonical fallback (Figure 2), an unsupported shape costs the user a transpilation attempt, never a correctness guarantee—and, in the spirit of Section 3.2, it becomes the next discriminating instance for the loop to absorb.

This catalog represents the current stable release. The transpiler remains under active development, continuously leveraging the agentic loop to safely synthesize, verify, and integrate new structural architectures as future models require them.

4 Computational Results

We evaluate the transpiler across two operational domains. Section 4.1 assesses the framework against three canonical benchmark formulations, establishing both strict structural equivalence against Pyomo (Section 4.1.1) and quantifying the resulting construction-time speedups (Section 4.1.2). Following these controlled experiments, Section 4.2 applies the agentic transpilation methodology to a large-scale epidemic model—the real-world use case that originally motivated this research.

4.1 Benchmark Models

We use three parameterized model families. Each is structured as a pure feasibility formulation—a set of decision variables and constraints without an objective—since our benchmark focuses on the computational cost of *constructing* the model architecture rather than solving it. Each family is specified once as a Pyomo model-building function and paired with a deterministic data generator. This ensures that the exact same Python specification reliably produces both the truncated instances required for verification and the massive instances required for benchmarking.

Supply–Demand. Given a set of sources I and destinations J, with supply $s_i \geq 0$ available at each source and demand $d_j \geq 0$ required at each destination, the decision variables $x_{ij} \geq 0$ denote the quantity shipped

from i to j. A feasible shipping plan satisfies:

$$\sum_{j \in J} x_{ij} \le s_i \quad \forall i \in I, \qquad \sum_{i \in I} x_{ij} \ge d_j \quad \forall j \in J, \qquad x_{ij} \ge 0.$$

An instance has $|I| \times |J|$ variables.

Multi-Commodity Network Flow. On a directed graph with nodes N and arcs $E \subseteq N \times N$, a set of commodities K must be routed simultaneously. For each node n, let $\delta^+(n)$ and $\delta^-(n)$ denote its outgoing and incoming arcs. Each arc (i, j) has capacity $u_{ij} \ge 0$ shared across commodities, and b_{nk} is the net flow of commodity k required at node n. The variables $x_{ijk} \ge 0$ carry commodity k on arc (i, j), and a feasible routing satisfies:

$$\sum_{k \in K} x_{ijk} \le u_{ij} \quad \forall (i,j) \in E, \qquad \sum_{j \in \delta^+(n)} x_{njk} \; - \sum_{i \in \delta^-(n)} x_{ink} = b_{nk} \quad \forall n \in N,\, k \in K,$$

with $x_{ijk} \ge 0$. The adjacency sets $\delta^+(\cdot)$ and $\delta^-(\cdot)$ are precomputed and supplied as indexed sets.

Bill-of-Materials. A production model links product builds to component purchases across periods. Over products P, components C, and periods T, let $P(c) \subseteq P$ be the products that use component c, let $a_{pc} \ge 0$ be the amount of c consumed per unit of p, and let $U_{pt} \ge 0$ be the capacity for building p in period t. The variables $y_{pt} \ge 0$ and $z_{ct} \ge 0$ denote the amount of product p built and component c bought in period t. A feasible plan satisfies:

$$y_{pt} \le U_{pt} \quad \forall p,\, t, \qquad z_{ct} = \sum_{p \in P(c)} a_{pc}\, y_{pt} \quad \forall c \in C,\, t \in T, \qquad y_{pt} \ge 0,\; z_{ct} \ge 0.$$

The coupling parameter a_{pc} is defined exclusively over the product–component pairs that actually occur, rather than over the dense Cartesian grid $P \times C$. Expressing this sparsity in the index structure is decisive for construction speed.

4.1.1 Verification of Equivalence

Before comparing construction times, we must rigorously confirm that the transpiled code and the Pyomo reference build the same model. Using the differential-testing procedure of Section 3.3, we compare the two solver-level models on many small instances drawn from the same specifications as the benchmarks. Because the data is an explicit argument and the reduced instances retain the indexing structure of the full model, Pyomo instantiation stays inexpensive and equivalence can be checked broadly rather than on a single case. Across 30 reduced instances of the three benchmark models—each at several sizes and three random seeds—and 21 additional formulations that isolate individual modeling features, every transpiled model proved structurally identical to its Pyomo reference.

4.1.2 Build-Time Performance

Having established equivalence, we now evaluate construction time. We compare four pipelines, all derived from the exact same Pyomo model-building function: standard Pyomo through `gurobi_persistent`; Pyomo with constraint and objective templatization compiled through Gurobi's matrix interface (`gurobi_direct`); the transpiler's initial `gurobipy-pandas` output; and its current output built from sparse coordinate-format matrices via `addMVar` and `addMConstr` (COO+MVar). Reported times measure model construction only and exclude the solve. To prevent Python's automatic memory management from distorting the measurements of the many-object models, each build is timed with the cyclic garbage collector disabled and a full collection forced beforehand. We impose a scaling threshold of 25 minutes. If a pipeline's construction time for a given instance exceeds this threshold, the run completes to establish the performance curve, but all larger instance

sizes for that pipeline are skipped and reported as timeouts. Table 6, Table 7, and Table 8 report the results in minutes.

Table 6: Build times for the Supply-Demand problem (in minutes). `gurobipy-pandas` denotes Claude Code's initial generated implementation using the `gurobipy-pandas` library; COO+MVar denotes the current Gurobi-native implementation using sparse COO matrices and `MVar`.

Size	Variables	Pyomo	Pyomo+Template	gurobipy-pandas	COO+MVar
2300^2	5,290,000	0.823	0.206	0.376	0.051
3300^2	10,890,000	1.955	0.410	0.766	0.107
4700^2	22,090,000	6.328	0.900	1.605	0.227
6500^2	42,250,000	70.699	2.241	3.188	0.506
9000^2	81,000,000	TIMEOUT	9.285	6.758	1.221

Table 7: Build times for the Network Flow problem.

Size	Variables	Pyomo	Pyomo+Template	gurobipy-pandas	COO+MVar
$n = 5000, k = 175$	5,250,000	1.142	0.302	0.470	0.087
$n = 7500, k = 225$	10,125,000	2.340	0.602	0.900	0.166
$n = 11000, k = 285$	18,810,000	8.707	1.272	1.741	0.322
$n = 16000, k = 360$	34,560,000	105.128	2.886	3.581	0.701
$n = 22000, k = 430$	56,760,000	TIMEOUT	5.849	6.627	1.361

Table 8: Build times for the Bill-of-Materials problem.

Size	Variables	Pyomo	Pyomo+Template	gurobipy-pandas	COO+MVar
$p = 10k, c = 20k, t = 80$	2,400,000	0.774	0.172	0.176	0.053
$p = 15k, c = 30k, t = 100$	4,500,000	1.479	0.337	0.328	0.098
$p = 22k, c = 44k, t = 120$	7,920,000	2.948	0.635	0.594	0.177
$p = 32k, c = 64k, t = 150$	14,400,000	15.080	2.439	1.181	0.351
$p = 45k, c = 90k, t = 180$	24,300,000	116.675	2.942	2.241	0.619

Standard Pyomo scales superlinearly across every model family and reliably exhausts the compute budget at the largest sizes. On the supply–demand model, construction rises from roughly one minute at 5.3 million variables to over seventy minutes at 42 million. The network-flow model behaves similarly. The cost is dominated by materializing one Python expression object for each variable and constraint, an overhead that persists regardless of the chosen backend.

Templatization narrows this gap substantially without closing it. By avoiding the per-constraint expression tree construction, it brings Pyomo to within a small constant factor of the native pipelines on the supply–demand and network-flow models—roughly four to eight times slower than COO+MVar, rather than orders of magnitude. The residual difference reflects the cost of instantiating one variable object per column before the matrix compiler runs. Because this $\Theta(V)$ cost is incurred per object, it remains proportional to model size, whereas the matrix pipelines avoid it entirely.

The bill-of-materials model illustrates a modeling pitfall severe enough to distort the comparison if left unchecked, and our handling of it is itself methodologically instructive. If the coupling parameter a_{pc} is declared over the dense grid $P \times C$ with a default value of zero, template compilation must resolve that default at every dense coordinate, and construction slows so sharply that templatized Pyomo becomes *slower* than standard Pyomo and fails outright at the larger sizes. Our first round of benchmarks did not control

for this and exhibited precisely that pathology. Recognizing it as an artifact of the formulation rather than of any pipeline, we declared a_{pc} over only the product–component pairs that occur—the sparse support described in Section 4.1—which produces an identical constraint matrix, as the equivalence test confirms, while restoring the expected scaling for every pipeline. The second round, controlled in this way, yields the times reported in Table 8. The lesson generalizes beyond this model: sparsity must be expressed in the index structure of the formulation, not left implicit in the values of a densely indexed parameter.

A natural concern is whether this reliance on sparse index structure merely displaces the cost into a data-preparation step that our timings exclude. It does not. Building the sparse sets—the adjacency lists $\delta^{\pm}(\cdot)$, the occurring product–component pairs $\{(p, c)\}$—is a single linear pass over information the modeler already holds; one cannot state which products consume which components, or which arcs exist, without it. At every benchmark size this preprocessing completed in well under a second, negligible beside even the fastest build. Nor is the discipline peculiar to our method: expressing sparsity in the index structure is precisely the hygiene that Pyomo's own templatization demands—a densely-defaulted parameter degrades it identically—and it is sound practice even for `gurobi_direct`. The transpiler inherits this convention rather than imposing it.

Across all three families, the transpiled matrix pipeline is fastest by a wide margin. COO+MVar constructs every instance in a small fraction of the time required by any Pyomo pipeline—on the order of a second per five million variables—and is the only method to complete the largest instance of each family within the budget. Because it assembles the sparse matrices directly and never creates per-index Python objects, its cost tracks the number of nonzeros rather than the number of modeling components. Together with the equivalence established in Section 4.1.1, this realizes the workflow of Figure 2: a model retains the readability of its Pyomo specification while being constructed at the speed of hand-written Gurobi matrix code, and without the standard trade-off between fast initial construction and fast iterative re-solving.

4.2 Case Study: Epidemic Model

References

[1] Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. Optimus-0.3: Using large language models to model and solve optimization problems at scale. *arXiv preprint arXiv:2407.19633*, 2024.

[2] Ali AhmadiTeshnizi, Wenzhi Gao, and Madeleine Udell. Optimus: Scalable optimization modeling with (mi) lp solvers and large language models. *arXiv preprint arXiv:2402.10172*, 2024.

[3] Anthropic. Agentic coding and persistent returns to expertise. `https://www.anthropic.com/research/claude-code-expertise`, June 2026. Accessed: 2026-07-06.

[4] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A Seshia, and Alvin Cheung. Verified code transpilation with llms. *Advances in Neural Information Processing Systems*, 37:41394–41424, 2024.

[5] Michael L Bynum, Gabriel A Hackebeil, William E Hart, Carl D Laird, Bethany L Nicholson, John D Siirola, Jean-Paul Watson, David L Woodruff, et al. *Pyomo-optimization modeling in python*, volume 67. Springer, 2021.

[6] Nicholas Carlini. Building a c compiler with a team of parallel claudes. `https://www.anthropic.com/engineering/building-c-compiler`, February 2026. Accessed: 2026-07-06.

[7] Hao Chen, Gonzalo E Constante-Flores, and Can Li. Diagnosing infeasible optimization problems using large language models. *INFOR: Information Systems and Operational Research*, 62(4):573–587, 2024.

[8] Junyan Cheng, Ankit Srivastava, Jessie Zeng, Milenko Drinic, and Jack W Stokes. Apeiron: A scalable llm-agentic framework for autonomous full-lifecycle demand-optimized application synthesis. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 3868–3899, 2026.

[9] Muhammad Farrukh, Baris Coskun, Tapti Palit, and Michalis Polychronakis. Safetrans: Llm-assisted transpilation from c to rust. In *Proceedings of the 1st Workshop on Code Translation, Transformation, and Modernization*, pages 30–37, 2026.

[10] William E Hart, Jean-Paul Watson, and David L Woodruff. Pyomo: modeling and solving mathematical programs in python. *Mathematical Programming Computation*, 3(3):219–260, 2011.

[11] Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruoqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 73(6):2986–3009, November 2025.

[12] Zangir Iklassov, Yali Du, Farkhad Akimov, and Martin Takáč. Self-guiding exploration for combinatorial problems. *Advances in Neural Information Processing Systems*, 37:130569–130601, 2024.

[13] Connor Lawless, Yingxi Li, Anders Wikum, Madeleine Udell, and Ellen Vitercik. Llms for cold-start cutting plane separator configuration. In *International Conference on the Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pages 51–69. Springer, 2025.

[14] Beibin Li, Konstantina Mellou, Bo Zhang, Jeevan Pathuri, and Ishai Menache. Large language models for supply chain optimization. *arXiv preprint arXiv:2307.03875*, 2023.

[15] Dong Li, Xujiang Zhao, Linlin Yu, Yanchi Liu, Wei Cheng, Zhengzhang Chen, Zhong Chen, Feng Chen, Chen Zhao, and Haifeng Chen. Solverllm: Leveraging test-time scaling for optimization problem via llm-guided search. *Advances in Neural Information Processing Systems*, 38:100028–100058, 2026.

[16] Laxmisha Rai, Smriti Khatiwada, Chunrao Deng, and Fasheng Liu. Cross-language code development with generative ai: A source-to-source translation perspective. In *2024 IEEE 7th International Conference on Electronic Information and Communication Technology (ICEICT)*, pages 562–565. IEEE, 2024.

[17] Rindra Ramamonjison, Haley Li, Timothy Yu, Shiqi He, Vishnu Rengan, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. Augmenting operations research with auto-formulation of optimization models from problem descriptions. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 29–62, 2022.

[18] Rindranirina Ramamonjison, Timothy Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, et al. Nl4opt competition: Formulating optimization problems based on their natural language descriptions. In *NeurIPS 2022 competition track*, pages 189–203. PMLR, 2023.

[19] Daniel Ramos, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. Batfix: Repairing language model-based transpilation. *ACM Transactions on Software Engineering and Methodology*, 33(6):1–29, 2024.

[20] Weiwei Sun, Shengyu Feng, Shanda Li, and Yiming Yang. Co-bench: Benchmarking language model agents in algorithm search for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33126–33134, 2026.

[21] Boris Weisfeiler and Andrei Leman. The reduction of a graph to canonical form and the algebra which appears therein. *Nauchno-Technicheskaya Informatsiya*, 2(9):12–16, 1968.

[22] Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. Chain-of-experts: When llms meet complex operations research problems. In *International Conference on Learning Representations*, volume 2024, pages 48519–48537, 2024.

[23] Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Shi Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. Optibench meets resocratic: Measure and improve llms for optimization modeling. In *International Conference on Learning Representations*, volume 2025, pages 24726–24759, 2025.

[24] Huigen Ye, Hua Xu, An Yan, and Yaoyang Cheng. Large language model-driven large neighborhood search for large-scale milp problems. In *Forty-second International Conference on Machine Learning*, 2025.

[25] Tom Zahavy. Position: Llms can't jump. In *Forty-third International Conference on Machine Learning Position Paper Track*, 2026.